

Lecture 16: Training transformer models

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain what it means to "train" a transformer model
2. Describe how training data is constructed from raw text
3. Understand cross-entropy loss and perplexity as training metrics
4. Describe the training loop and key optimization techniques
5. Distinguish between foundation models, instruct-tuned models, and fine-tuned models

Where we left off

Recap from lecture 15

Last time we saw the transformer as a function:

$$\text{Transformer}(X, \theta) \rightarrow Y$$

We traced data from raw text through tokenization, embedding, attention, and feed-forward layers. But we treated the parameters θ as given.

Today's question

Where do the parameters θ come from? How does the model *learn* to predict the next token?

What does "training" mean?

Training a language model

Training is the process of finding parameters θ that make the model's predictions match the data. Given a large corpus of text, we want:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

where $\mathcal{L}$ is a **loss function** that measures how wrong the model's predictions are.

Intuition

Imagine the model is a student learning to finish sentences. We show it millions of sentences, and every time it guesses the next word wrong, we nudge its parameters to make a better guess next time.

The language modeling objective

Next-token prediction

Given a sequence of tokens $x_1, x_2, \dots, x_{t-1}$, the model predicts a probability distribution over the vocabulary for the next token x_t :

$$P(x_t \mid x_1, x_2, \dots, x_{t-1}; \theta)$$

The goal is to maximize the probability of the *actual* next token across the entire training corpus.

This is self-supervised learning

No human labels needed! The "label" is simply the next word in the text. This is why we can train on virtually unlimited data from the internet.

From raw text to training data

How training data is constructed

Training data is built from raw text in three steps:

1. **Tokenize** every document in the corpus independently
2. **Concatenate** all token sequences into one long stream (optionally separated by `<EOS>` tokens)
3. **Chunk** the stream into fixed-length blocks of `block_size` tokens (e.g., 1024 or 2048)

Each chunk becomes one training example. Leftover tokens shorter than `block_size` are dropped.

Why not pad individual documents?

Padding wastes compute — every `[PAD]` token is a wasted FLOP. Concatenation ensures every token in every batch is a real training signal. This is the standard approach used by HuggingFace, GPT-2/3, LLaMA, and most modern language models.

The causal attention mask

How one chunk trains the whole model

Within each chunk, the **causal attention mask** (a lower-triangular matrix) ensures position t can only attend to positions $1, 2, \dots, t$. This means every position simultaneously predicts its next token — one forward pass through a chunk of length L produces L training signals.

Labels are simply the input shifted right by one position. The model handles this internally during the forward pass.

Intuition

Think of it like a classroom where every student takes the same test at the same time, but each student can only see the questions before theirs. Student 1 sees nothing and guesses the first word. Student 2 sees word 1 and predicts word 2. Student L sees all previous words and predicts the last. All L students learn simultaneously from a single test.

Cross-entropy loss

The loss function

Cross-entropy loss measures how different the model's predicted distribution $\hat{y}$ is from the true distribution y (a one-hot vector for the correct token):

$$\mathcal{L} = - \sum_{i=1}^V y_i \log(\hat{y}_i)$$

Since y is one-hot (only the correct token has $y_i = 1$), this simplifies to:

$$\mathcal{L} = -\log(\hat{y}_c)$$

where c is the index of the correct next token.

Understanding cross-entropy

Why negative log probability?

- If the model assigns probability 1.0 to the correct token: $-\log(1.0) = 0$ (no loss!)
- If the model assigns probability 0.5: $-\log(0.5) = 0.69$
- If the model assigns probability 0.01: $-\log(0.01) = 4.6$ (high loss!)

The loss penalizes confident *wrong* predictions heavily and rewards confident *correct* predictions.

Concrete example

```
1 Vocabulary: [the, cat, sat, on, mat] (V=5)
2 True next token: "sat" → y = [0, 0, 1, 0, 0]
3
4 Model prediction:  $\hat{y} = [0.1, 0.2, 0.5, 0.15, 0.05]$ 
5 Loss =  $-\log(0.5) = 0.69$ 
6
7 Better prediction:  $\hat{y} = [0.05, 0.05, 0.85, 0.03, 0.02]$ 
8 Loss =  $-\log(0.85) = 0.16$  ← much lower!
```

Perplexity

Perplexity: a more intuitive metric

Perplexity is the exponential of the average cross-entropy loss across a sequence:

$$\text{PPL} = e^{\mathcal{L}} = e^{-\frac{1}{N} \sum_{t=1}^N \log P(x_t | x_{<t})}$$

A perplexity of k means the model is, on average, as uncertain as if it were choosing uniformly among k options.

Perplexity in practice

- **Model comparison:** Lower PPL = better model. GPT-2 Small (PPL ≈ 27) vs. GPT-2 XL (PPL ≈ 17) vs. GPT-3 (PPL ≈ 11)
- **Training monitoring:** Plot PPL over training steps; it should decrease. Spikes indicate instability.
- **Evaluation:** Standard metric on held-out test sets (e.g., WikiText-103)

Limitations of perplexity

Only measures token-level prediction accuracy — doesn't capture fluency, coherence, or factual correctness. A model with low PPL can still hallucinate or generate toxic content. Can't compare models with different vocabularies (different PPL scales), and values are domain-dependent.

The training loop

Four steps, repeated millions of times

1. **Forward pass:** feed a batch of text through the model to get predictions
2. **Compute loss:** compare predictions to actual next tokens using cross-entropy
3. **Backward pass:** compute gradients $\nabla_{\theta}\mathcal{L}$ via backpropagation
4. **Update parameters:** adjust θ to reduce the loss

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

where η is the **learning rate** — how big a step we take.

Scale matters

GPT-3 has 175 billion parameters. Each training step updates *all* of them. Training took ~\$4.6 million in compute costs and processed ~300 billion tokens. GPT-4 likely cost over \$100 million to train, and GPT-5 (trillions of parameters) was even more expensive to train. Energy consumption and environmental impact are significant concerns at this scale.

Optimization and regularization

Key techniques for training transformers

- **Stochastic gradient descent (SGD):** Use random mini-batches instead of the entire dataset. Faster and noisier, but works well in practice.
- **AdamW optimizer:** Gives each parameter its own adaptive learning rate based on gradient history. Adds momentum (smooths updates) and weight decay (prevents overfitting). The standard choice for transformers.
- **Learning rate scheduling:** Start with a warmup phase (gradually increase LR), then decay with a cosine or linear schedule. Prevents instability early in training.
- **Gradient clipping:** Cap gradient magnitude to prevent "exploding gradients" from destroying training progress. Typical max norm = 1.0.

Further reading

Loshchilov & Hutter (2019, ICLR) "Decoupled Weight Decay Regularization" — The AdamW optimizer paper.

Vaswani et al. (2017, NeurIPS) "Attention Is All You Need" — Introduced warmup scheduling for transformers.

Scaling laws

Bigger models + more data = better performance

Language model performance follows predictable **power laws**: smooth, straight lines on log-log plots when you increase model size, dataset size, or compute budget.

- **Kaplan et al. (2020)**: Discovered these relationships across seven orders of magnitude
- **Hoffmann et al. (2022, "Chinchilla")**: Showed optimal training balances model size and data — a smaller model trained on more data can match a larger undertrained model (Chinchilla 70B matched GPT-3 175B)

Key takeaway: you can predict model performance before training by running small-scale experiments first.

Further reading

Kaplan et al. (2020, *arXiv*) "Scaling Laws for Neural Language Models"

Hoffmann et al. (2022, *arXiv*) "Training Compute-Optimal Large Language Models" (Chinchilla)

Training with HuggingFace

Pre-training a language model

```
1  from transformers import AutoModelForCausalLM, AutoTokenizer
2  from transformers import Trainer, TrainingArguments
3  from datasets import load_dataset
4
5  model = AutoModelForCausalLM.from_pretrained("gpt2")
6  tokenizer = AutoTokenizer.from_pretrained("gpt2")
7
8  dataset = load_dataset("wikitext", "wikitext-2-raw-v1")
9  def tokenize(examples):
10     return tokenizer(examples["text"], truncation=True, max_length=512)
11  tokenized = dataset.map(tokenize, batched=True)
12
13  args = TrainingArguments(
14     output_dir="./results", num_train_epochs=3,
15     per_device_train_batch_size=8, learning_rate=5e-5,
16     warmup_steps=500, weight_decay=0.01,
17  )
18  trainer = Trainer(model=model, args=args, train_dataset=tokenized["train"])
19  trainer.train()
```


From pre-training to deployment

Three stages of model development

1. **Pre-training** → **Foundation model**: Train on a massive general text corpus (books, web, code). The model learns language patterns, world knowledge, and reasoning. Expensive (\$millions), done once. Examples: GPT-3 base, LLaMA, Mistral.
2. **Instruction tuning + RLHF** → **Instruct model**: Fine-tune the foundation model on instruction-following data and human preference feedback. The model learns to be helpful, harmless, and honest. This is what makes ChatGPT different from raw GPT-4. Examples: ChatGPT, Claude, Gemini.
3. **Task-specific fine-tuning** → **Specialized model**: Further fine-tune on domain data for a specific application. Cheap, fast, and highly effective. Examples: medical diagnosis, legal analysis, code generation for a specific codebase.

Why instruction tuning matters

Foundation models are powerful but unruly

A foundation model has vast knowledge but no "manners" — it will complete any text prompt, including harmful or nonsensical ones. Instruction tuning teaches the model to follow instructions, answer questions helpfully, and refuse harmful requests.

RLHF (Reinforcement Learning from Human Feedback): Humans rank model outputs, and the model is trained to prefer higher-ranked responses. This is the key technique behind ChatGPT, Claude, and other assistants.

Further reading

Ouyang et al. (2022, *NeurIPS*) "Training language models to follow instructions with human feedback" — The InstructGPT paper that launched the instruction-tuning revolution.

Bai et al. (2022, *arXiv*) "Constitutional AI: Harmlessness from AI Feedback" — An alternative to human feedback.

Task-specific fine-tuning

Adapting a model to your task

Take a pre-trained (or instruct-tuned) model and train it further on task-specific data:

- Only needs small datasets (hundreds to thousands of examples)
- Uses tiny learning rates (10-100x smaller than pre-training) to preserve existing knowledge
- Training takes minutes to hours, not weeks

Why does it work?

The pre-trained model already understands language, facts, and reasoning. Fine-tuning just teaches it the *format* and *focus* of your task — like an expert learning a new specialty. Fine-tuning on 1,000 labeled examples typically outperforms training from scratch on 100,000 examples.

Fine-tuning with HuggingFace

Fine-tuning for sentiment classification

```
1  from transformers import AutoModelForSequenceClassification
2  from transformers import Trainer, TrainingArguments
3  from datasets import load_dataset
4
5  model = AutoModelForSequenceClassification.from_pretrained(
6      "bert-base-uncased", num_labels=2
7  )
8  dataset = load_dataset("imdb")
9
10 args = TrainingArguments(
11     output_dir="./sentiment-model", num_train_epochs=3,
12     per_device_train_batch_size=16, learning_rate=2e-5,
13     warmup_ratio=0.1, weight_decay=0.01,
14     evaluation_strategy="epoch",
15 )
16 trainer = Trainer(
17     model=model, args=args,
18     train_dataset=dataset["train"], eval_dataset=dataset["test"],
19 )
```

Practical training advice

Rules of thumb

1. **Start small:** Test your pipeline with a small model before scaling up
2. **Learning rate:** Fine-tuning uses 10-100x smaller learning rates than pre-training (typically $1e-5$ to $5e-5$)
3. **Epochs:** Fine-tuning needs only 2-5 epochs; pre-training typically makes 1-2 passes over the data since the dataset is so large
4. **Batch size:** Larger batches give more stable gradients but require more memory. Use gradient accumulation if your GPU is too small
5. **Evaluation:** Always hold out a validation set and monitor for overfitting

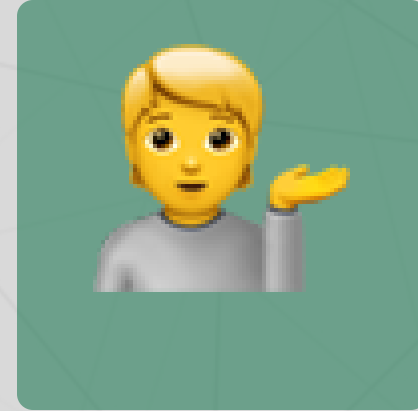
Questions?



Email



Discord



Office hours

Up next...

Retrieval Augmented Generation (RAG): giving language models access to external knowledge!